

Methods 3: Multilevel Statistical Modeling and Machine Learning

Week 48: *Unsupervised classification*
November 26, 2025

The course plan

Week 37:

Lesson 0: What is it all about?

Class 0: Setting up R and Python — and recollection of the general linear model (Lau)

Week 38:

Lesson 1: Multilevel linear regression

Class 1: Modelling subject level effects – and how do they differ from group level effects? (Lydia)

Week 39:

Lesson 2: Beyond linear regression — link functions

Class 2: What to do when the response variable is not continuous? (Lau)

Week 40:

Lesson 3: Evaluating generalised linear mixed models — and the spectre of collinearity

Class 3: Code review (Lydia)

Week 41:

Lesson 4: Explanation versus prediction

Class 4: Summarising mixed models (Lydia)

Week 42:

no teaching (go fetch potatoes instead)

Week 43:

Lesson 5: Mid-way evaluation and machine learning introduction

Class 5: Getting Python running (Lau)

Week 44:

Lesson 6: Linear and logistic regression revisited (machine learning)

Class 6: Categorizing responses based on informed guesses (Lau)

Week 45:

Lesson 7: Model evaluation and hyperparameter tuning

Class 7: Code review (Lydia)

Week 46:

Lesson 8: Dimensionality reduction — principled component analysis (PCA)

Class 8: What to do with very rich data? (Lau)

Week 47:

Lesson 9: Neural networks

Class 9: Code review (Lau)

Week 48:

Lesson 10: Unsupervised classification

Class 10: Create video explainers of central concepts (shared resource) (Lydia)

Week 49:

Lesson 0 again: What was it all about? and final evaluation

Class 11: Summarising machine learning (Lydia)

Week 50:

Student presentations:

Class 12: Work on portfolios: Ask anything! (Lydia & Lau)

Recap

NEURAL NETWORKS

- Neural networks combine many pieces we have seen so far
 - Linear and logistic regression
 - Regularisation
 - Gradient descent
- The main difference is that we apply the steps several times
 - We still calculate net input (z), also known as the linear predictor ($X\beta$)
 - We still apply an activation function $\phi(z)$ to get data back on an interpretable scale
 - We still apply a quantiser function to make a decision ($\text{np.argmax}(z)$)
- And that we use backpropagation to optimise fitting

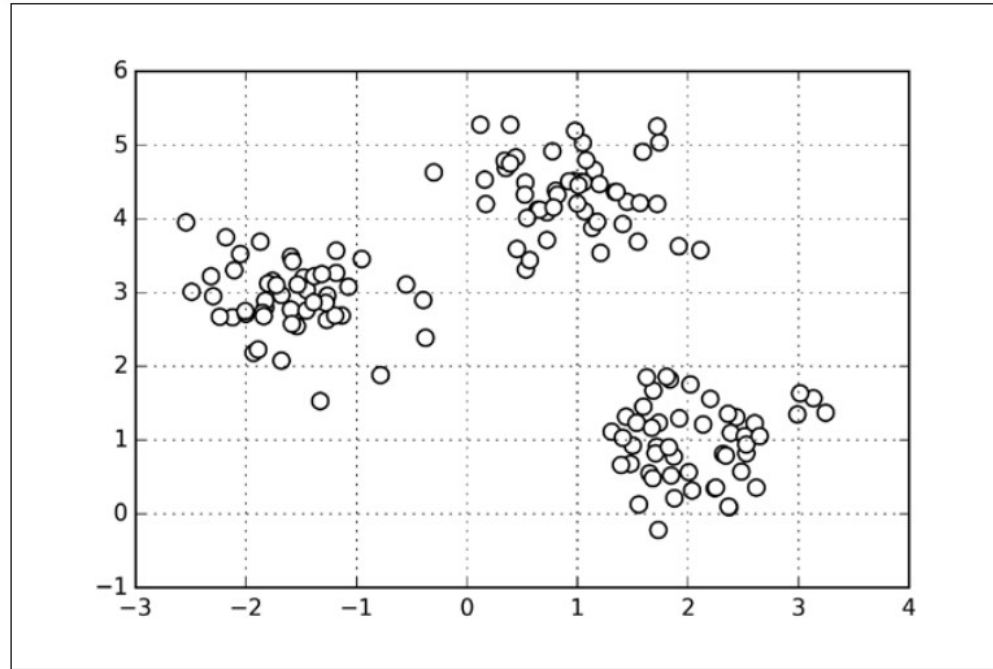
Learning goals and outline

Unsupervised classification

- 1) Understanding the *k*-means algorithm
- 2) Understanding the difference between unsupervised and supervised learning
- 3) Being able to build your own *k*-means algorithm

Unsupervised?

Unlabelled data





By 丁志仁 - Own work (imitated from File:ChatGPT-Logo.png),
CC BY-SA 4.0,
<https://commons.wikimedia.org/w/index.php?curid=145322793>

Use cases

examples of k-means being used in businesses

K-means clustering is a powerful machine learning algorithm used for grouping data into clusters based on similarity. Businesses across industries employ k-means clustering for various purposes, such as understanding customer behavior, optimizing operations, and segmenting data. Here are some real-world examples:

8. Social Media Analysis

Industry: Technology and Social Media

Example:

Social media platforms use k-means clustering to group users based on their interests, activity patterns, or content engagement. This helps recommend relevant content or friends.

1. Customer Segmentation

Industry: Retail and E-commerce

Example:

A retailer can use k-means to segment customers into groups based on their purchase history, spending habits, or demographics. This helps in:

- Personalizing marketing campaigns.
- Identifying high-value customers for loyalty programs.
- Tailoring product recommendations.

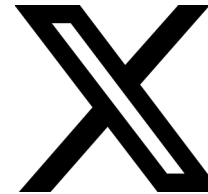


<https://www.magasin.dk/>



<https://medlem.coop.dk>

facebook





Use cases

By 丁志仁 - Own work (imitated from File:ChatGPT-Logo.png),
CC BY-SA 4.0,
<https://commons.wikimedia.org/w/index.php?curid=145322793>

examples of k-means being used in businesses

 K-means clustering is a powerful machine learning algorithm used for grouping data into clusters based on similarity. Businesses across industries employ k-means clustering for various purposes, such as understanding customer behavior, optimizing operations, and segmenting data. Here are some real-world examples:

10. Healthcare Analytics

Industry: Healthcare

Example:

Hospitals use k-means to group patients based on symptoms or medical history. This helps in personalized treatment planning and predicting disease risk.

k -means procedure

- 1) Randomly pick k centroids from the sample points as initial cluster centers. (*initialisation*)
- 2) Assign each sample to the nearest centroid $\mu^{(j)}, j \in \{1, \dots, k\}$. (*assignment*)
- 3) Move the centroids to the center of the samples that were assigned to it. (*assignment*)
- 4) Repeat the steps 2 and 3 until the cluster assignment does not change or a user-defined tolerance or a maximum number of iterations is reached. (*optimisation*)

Optimisee

WITHIN-CLUSTER SUM OF SQUARED ERRORS

$$SSE = \sum_{i=1}^n \sum_{j=1}^k w^{(i, j)} \left\| \mathbf{x}^{(i)} - \boldsymbol{\mu}^{(j)} \right\|_2^2$$

Discussion point:

What does this equation do?

i : sample index

j : cluster index

n : n samples

k : n clusters

$\boldsymbol{\mu}$: centroid for cluster j

$w^{(i, j)} = 1$, if $\mathbf{x}^{(i)}$ in cluster j

$w^{(i, j)} = 0$, if $\mathbf{x}^{(i)}$ *not* in cluster j

Optimisee

WITHIN-CLUSTER SUM OF SQUARED ERRORS

$$SSE = \sum_{i=1}^n \sum_{j=1}^k w^{(i,j)} \left\| \mathbf{x}^{(i)} - \boldsymbol{\mu}^{(j)} \right\|_2^2$$

i : sample index

j : cluster index

n : n samples

k : n clusters

$\boldsymbol{\mu}$: centroid for cluster j

$w^{(i,j)} = 1$, if $\mathbf{x}^{(i)}$ in cluster j

$w^{(i,j)} = 0$, if $\mathbf{x}^{(i)}$ *not* in cluster j

Discussion point:

What preprocessing step for $\mathbf{x}$ is necessary for minimising SSE ?

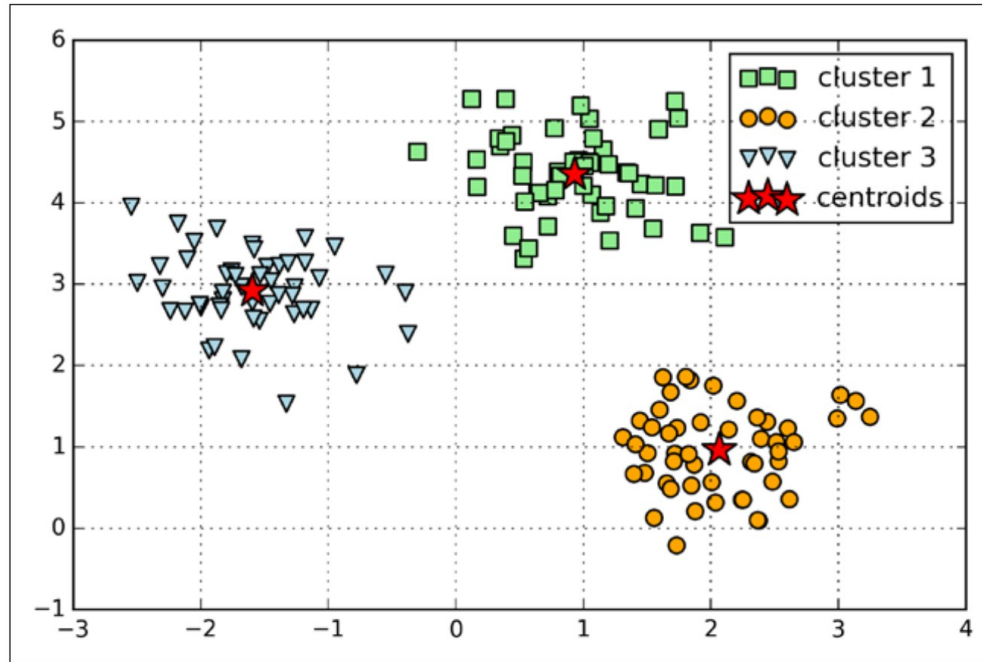
Running over each cluster

Running over each sample

Minimise distance from centroid

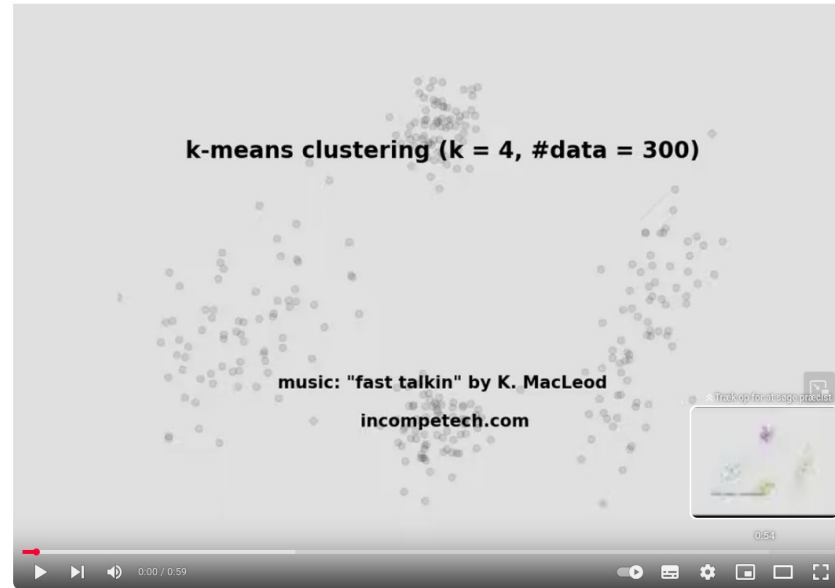
... but only if it is part of cluster

Labelled by k -means



A cute, little video

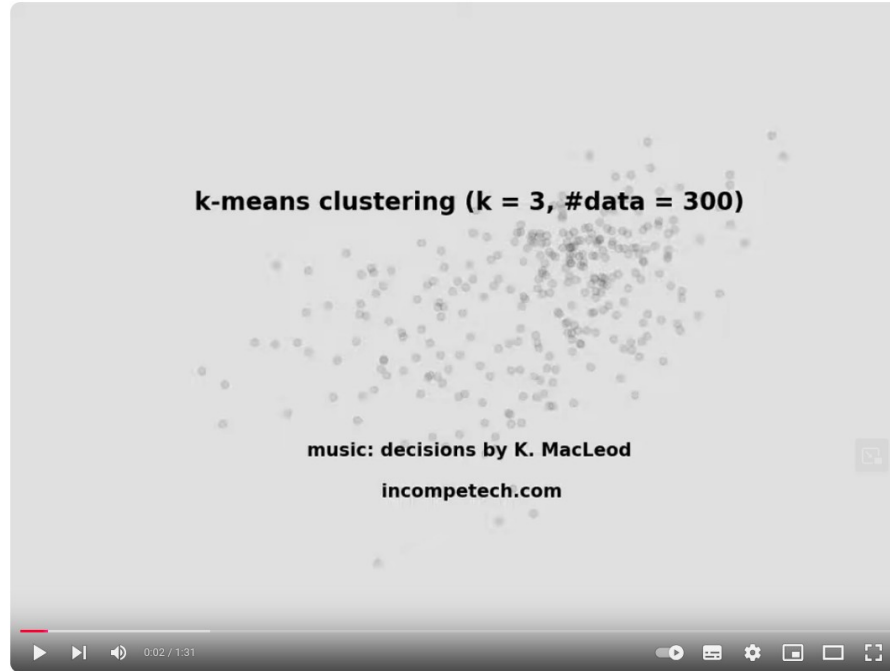
k-means clustering; by bitLectures: <https://www.youtube.com/watch?v=5l3Ei69l40s&t=59s>



Features and issues – k -means

- When converged, clusters may reveal patterns in data
- Some clusters may end up being empty
- k must be chosen *a priori* without any guiding information
 - elbow and silhouette plots may help
- May converge slowly or with vastly different results if *centroids* are chosen unluckily, e.g. too close to one another
 - k -means++ may help, i.e. it places initial centroids far from one another
- Clusters don't overlap
 - You can alter the “hardness” of the cluster

And another one



k-means clustering: effect of different initializations ; by bitLectures; <https://www.youtube.com/watch?v=9nKfViAfajY>

Hard vs. soft cluster

AND FUZZINESS (m)

$$\begin{bmatrix} \mu^{(1)} \rightarrow 0 \\ \mu^{(2)} \rightarrow 1 \\ \mu^{(3)} \rightarrow 0 \end{bmatrix}$$

$$\begin{bmatrix} \mu^{(1)} \rightarrow 0.1 \\ \mu^{(2)} \rightarrow 0.85 \\ \mu^{(3)} \rightarrow 0.05 \end{bmatrix}$$

$$J_m = \sum_{i=1}^n \sum_{j=1}^k w^{[m]^{i,j}} \left\| \mathbf{x}^{(i)} - \boldsymbol{\mu}^{(j)} \right\|_2^2, \quad [m] \in [1, \infty)$$

$w^{(i,j)} \in [0, 1]$: can be any probability (not binary)

How to find k ?

Discussion point:

Why can we not just treat k as a hyperparameter,
as we did three weeks ago?
(Similarly to what we did with $n_components$ in
PCA?)

We have to rely on *intrinsic metrics*

WE HAVE SEEN INTRINSIC METRICS BEFORE

$$R^2 = 1 - \frac{\sum_{i=1}^n (y_i - \hat{y}_i)^2}{\sum_{i=1}^n (y_i - \mu_y)^2}$$

$$\text{deviance} = -2l(\hat{\theta})$$

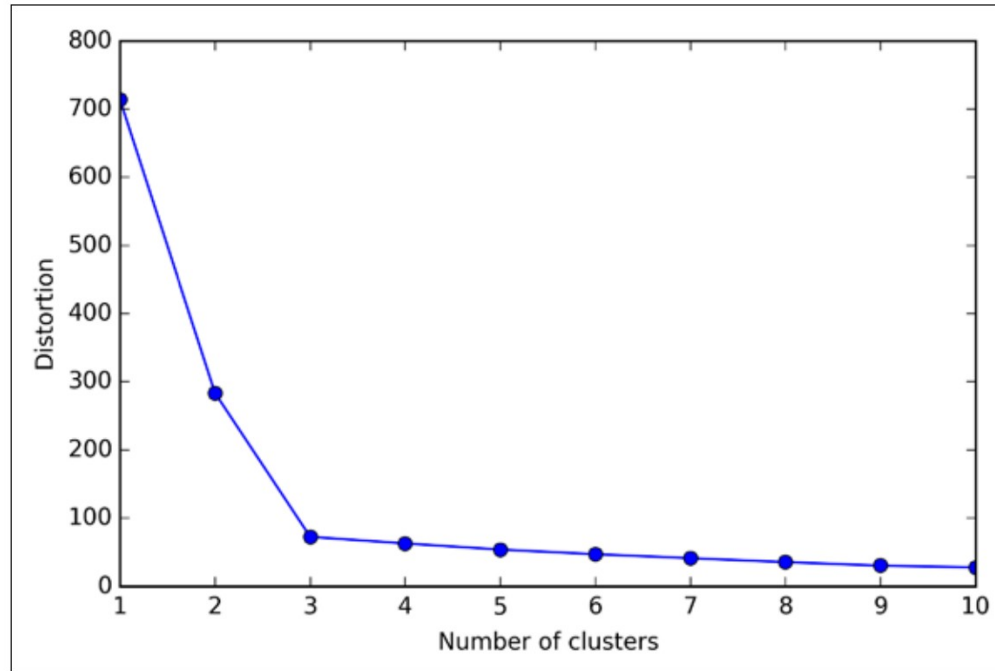
$$\text{AIC} = \text{deviance} + 2k$$

k : number of predictors

$$\bar{R}^2 = 1 - (1 - R^2) \frac{n-1}{n-p-1}$$

Elbow plot

DISTORTION = SSE



Discussion points:

Where is the elbow?

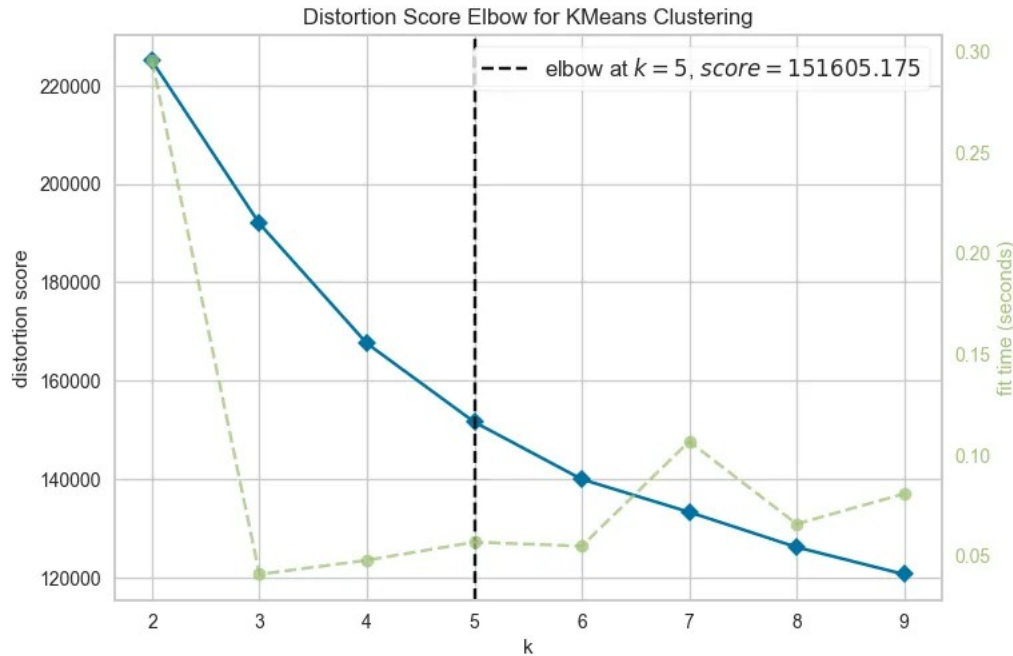
What may be the problem in choosing the elbow in a real (non-simulated) dataset?

How does this relate to a similar problem one might have with PCA?

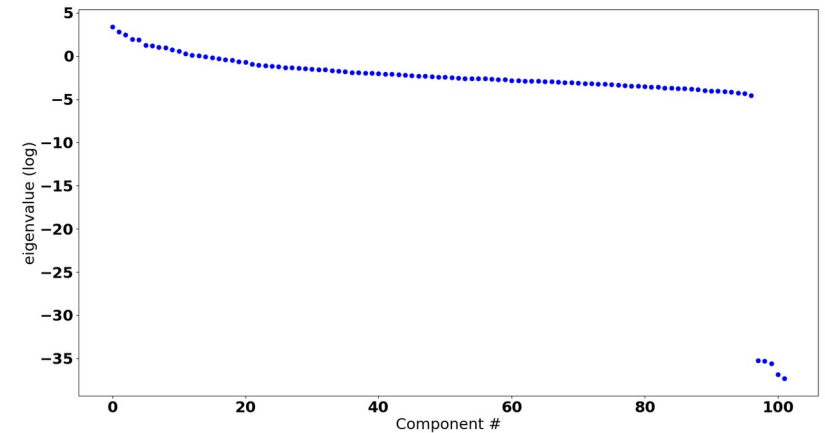
Customer segmentation

<https://medium.com/towards-data-science/mastering-customer-segmentation-with-llm-3d9008235f41>

ELBOW OR STRETCHED ARM?



SIMILAR EXAMPLE WITH PCA



Silhouette plots

may help inform the decision giving the *cohesiveness* and the *separation* of clusters

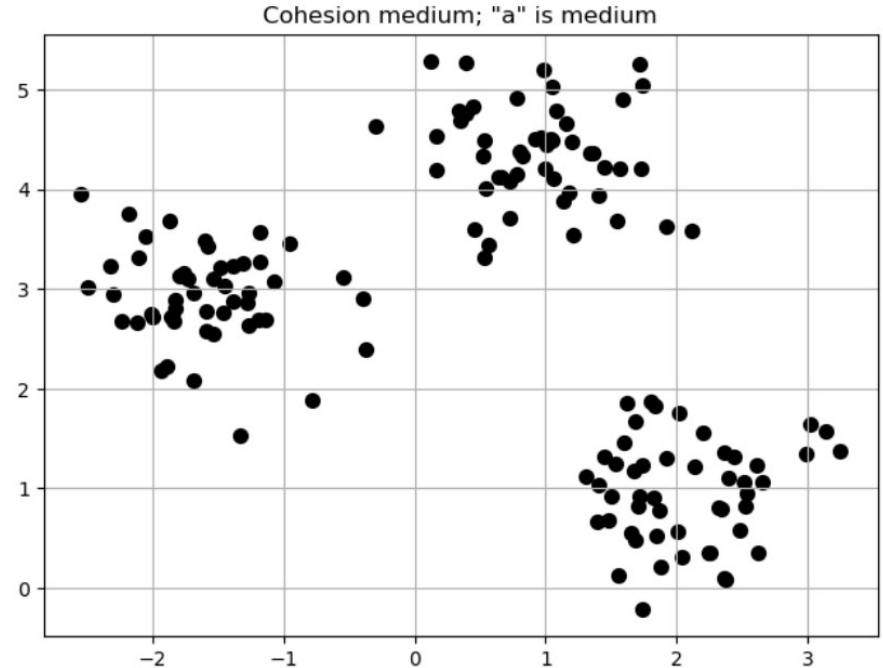
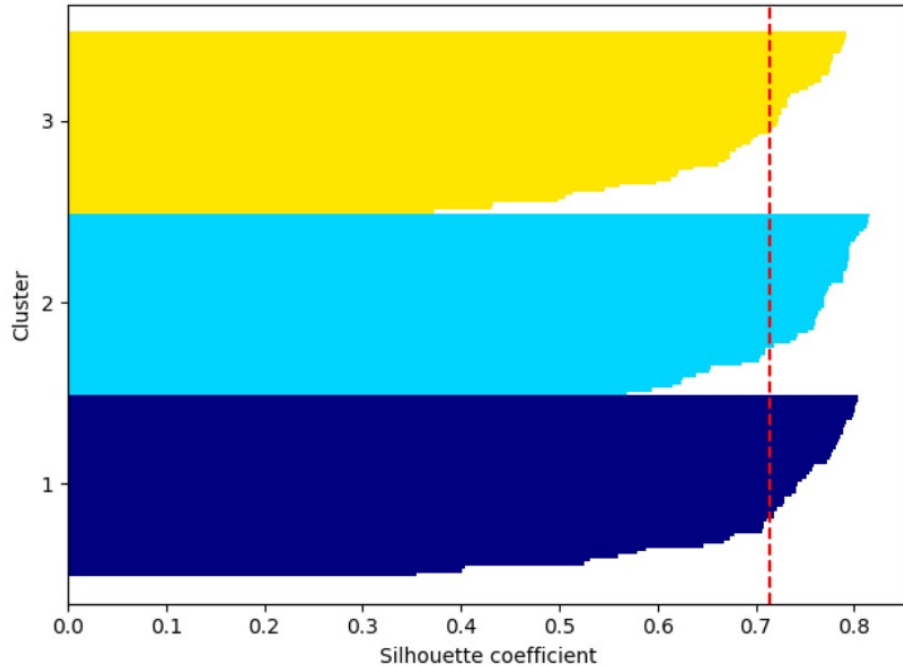
Silhouette plots

- 1) Calculate the cluster cohesion $a^{(i)}$ as the average distance between a sample $x^{(i)}$ and all other points in the same cluster.
- 2) Calculate the cluster separation $b^{(i)}$ from the next closest cluster as the average distance between the sample $x^{(i)}$ and all samples in the nearest cluster.
- 3) Calculate the silhouette $s^{(i)}$ as the difference between cluster cohesion and separation divided by the greater of the two, as shown here:

$$s^{(i)} = \frac{b^{(i)} - a^{(i)}}{\max\{b^{(i)}, a^{(i)}\}} \quad s^{(i)} \in [-1, 1]$$

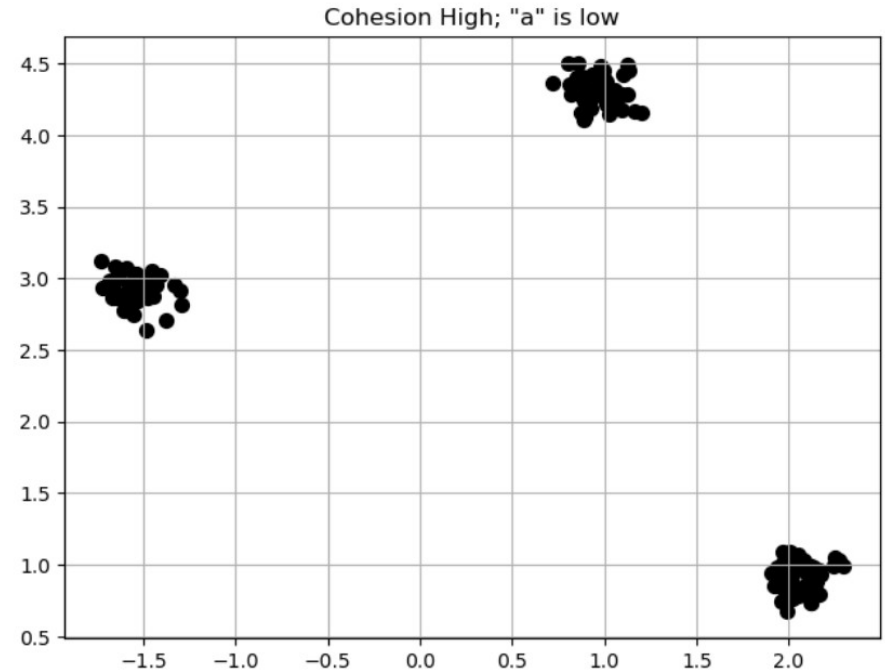
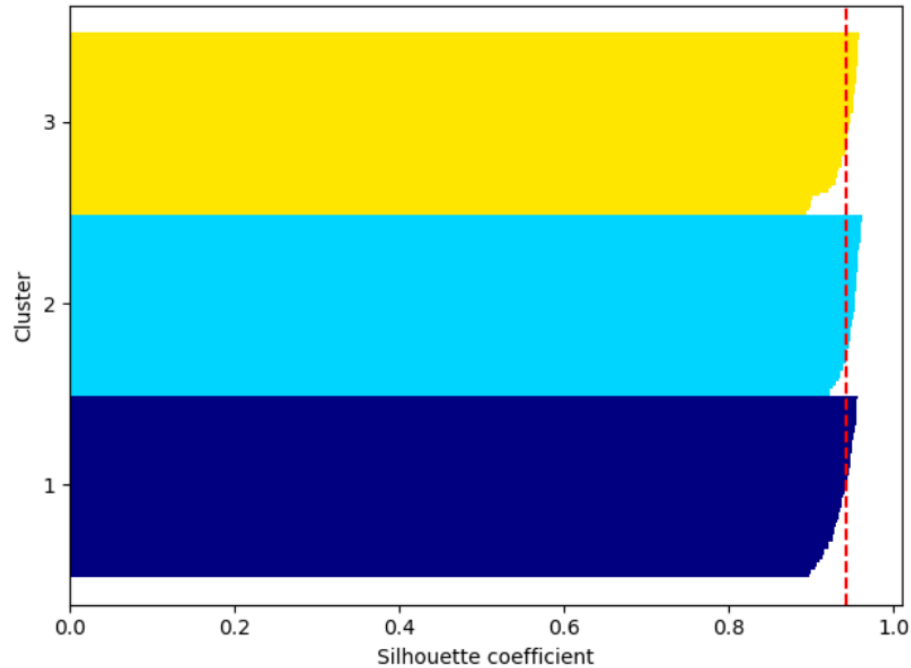
Medium cohesion

RED LINE: AVERAGE SCORE



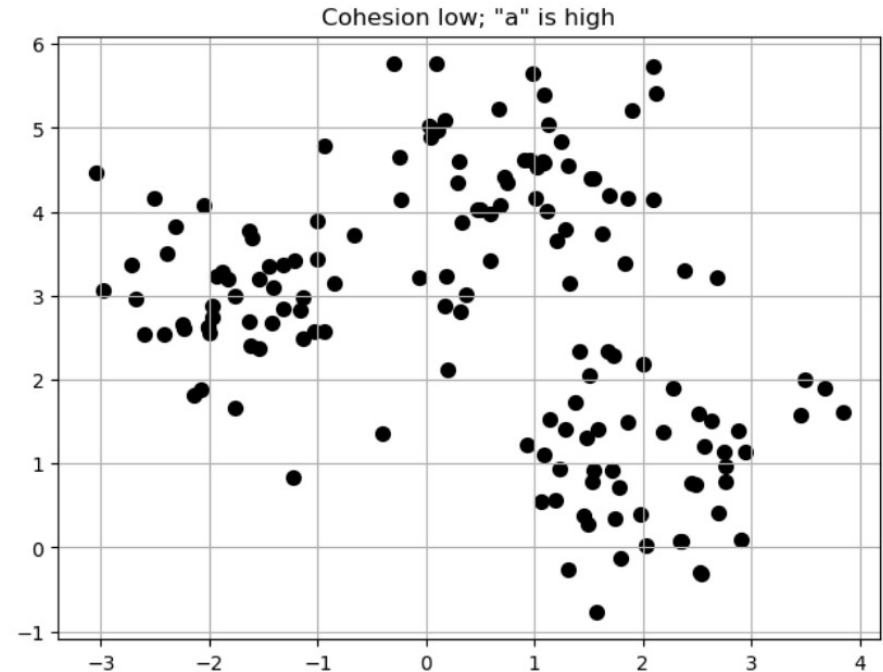
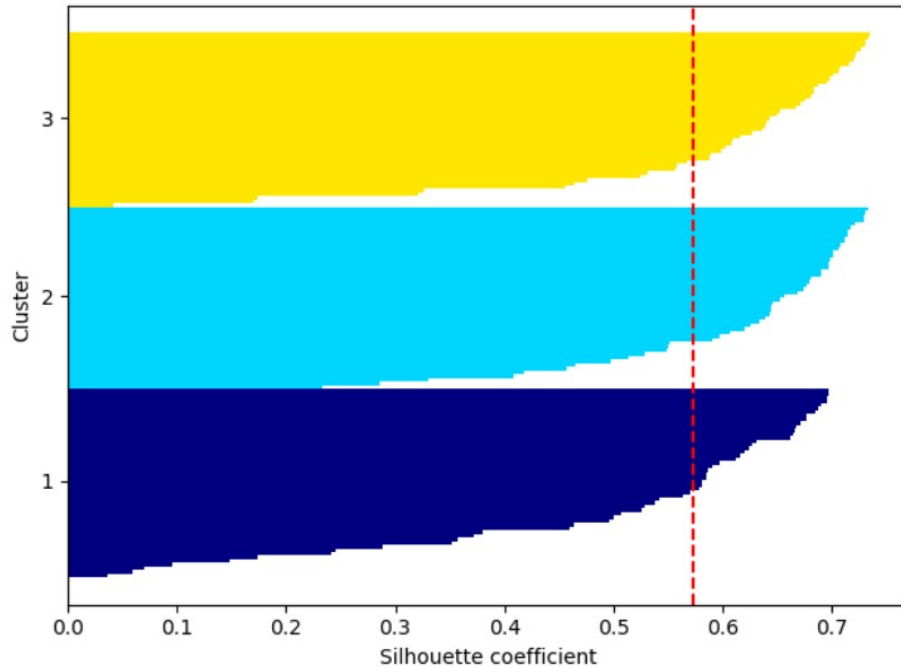
High cohesion

RED LINE: AVERAGE SCORE



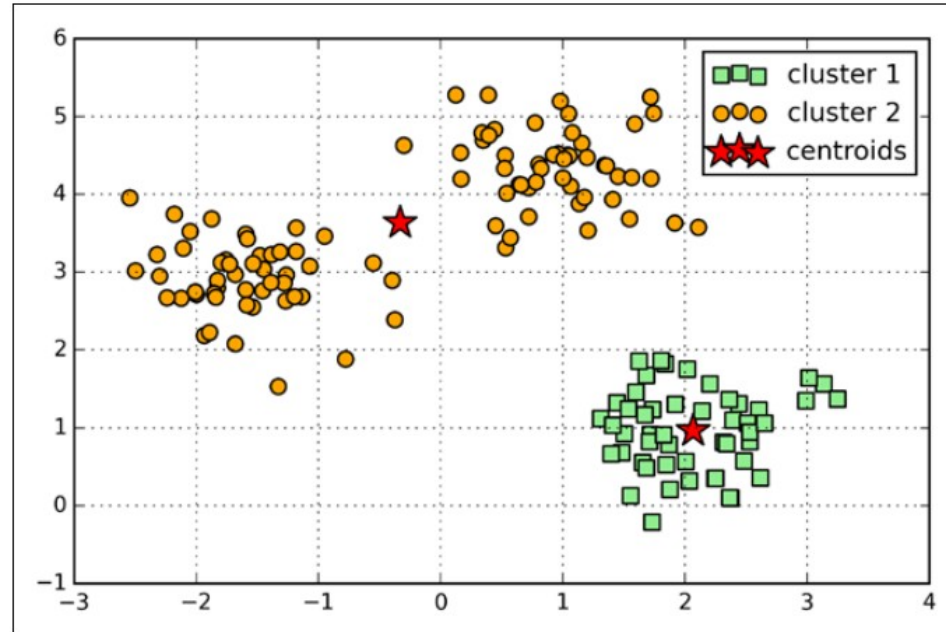
Low cohesion

RED LINE: AVERAGE SCORE



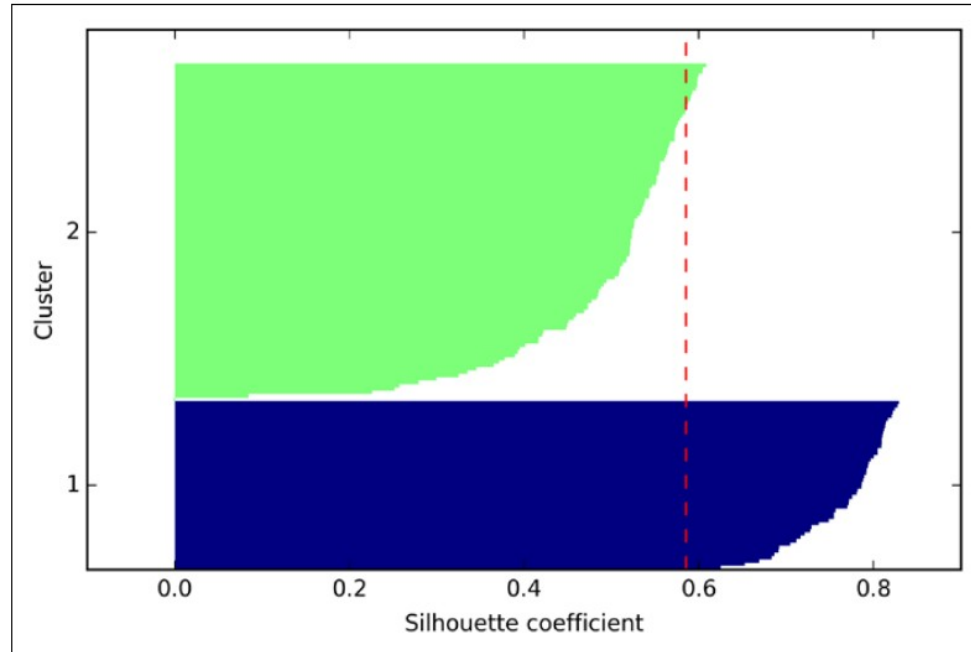
Doing it wrongly on purpose

$k=2$



Doing it wrongly on purpose

$k=2$



Good and bad silhouettes

- Good silhouettes
 - Long and slender
 - Uniform
 - High scores
- Bad silhouettes
 - Wide bars
 - Irregular bars
 - Low scores

Interim summary

- We can separate data into meaningful clusters, even when they don't have labels
 - Purely data-driven
 - These clusters may be meaningful, and we can label them, e.g.:
“interested in 80's-movies”, “will likely buy shoes every month”, “may have diabetes” etc.
- With no ground truth, we have to use internal metrics, e.g. elbow and silhouette plots to evaluate fits
- Works best for spherical clusters that are well separated



What other choices are there?

NOT COVERED TODAY

By 丁志仁 - Own work (imitated from File:ChatGPT-Logo.png),
CC BY-SA 4.0,
<https://commons.wikimedia.org/w/index.php?curid=145322793>

what kinds of unsupervised classification does scikit-learn allow for?

Choosing the Right Algorithm

- **Well-Separated, Spherical Clusters:** K-Means, GMM.
- **Arbitrary Shapes or Noise:** DBSCAN, OPTICS.
- **Hierarchical Relationships:** Agglomerative Clustering.
- **Non-Euclidean Data:** Spectral Clustering.
- **Large Datasets:** Birch, MiniBatchKMeans.

These methods provide flexibility for handling a wide variety of clustering problems and data characteristics.

Instead ...

You'll get to work ...



You'll build your own k -means algorithm

```
import numpy as np
import sys

class kmeans(object):

    def __init__(self, k=3, max_iter=100, tol=1e-4):
        self.max_iter      = max_iter
        self.tol            = tol
        self.k              = k
        self.centroids_     = None # add these
        self.centroid_labels_ = None # add these
        self.cost_          = list() # empty list
```

Remember the four steps

- 1) Randomly pick k centroids from the sample points as initial cluster centers. (*initialisation*)
- 2) Assign each sample to the nearest centroid $\mu^{(j)}, j \in \{1, \dots, k\}$. (*assignment*)
- 3) Move the centroids to the center of the samples that were assigned to it. (*assignment*)
- 4) Repeat the steps 2 and 3 until the cluster assignment does not change or a user-defined tolerance or a maximum number of iterations is reached. (*optimisation*)

Steps 1 & 2

```
def initialise_centroids(self, X):  
    ## 1) Randomly pick k centroids from the sample points  
    ## as initial cluster centers  
    ## have a look at np.random.choice to choose random starters  
  
    initial_centroids = None # change this  
    return initial_centroids
```

```
def assign_to_centroids(self, X, centroids):  
    ## 2) Assign each sample to the nearest centroid  $\mu(j)$ ,  $j \in \{1, \dots, k\}$   
    ## have a look at np.linalg.norm to calculate the distance from which  
    ## you can get the centroid label  
  
    n_samples = X.shape[0]  
    centroid_labels = np.zeros(n_samples) ## fill in labels (0, 1, 2)  
  
    return centroid_labels
```

Step 3 and cost

```
def move_centroid_to_new_centres(self, X, centroid_labels):  
    ## 3) Move the centroids to the center of the samples  
    ## that were assigned to it  
  
    new_centroids = np.zeros(shape=(self.k, X.shape[1])) # fill coordinates  
  
    return new_centroids
```

```
def calculate_cost(self, X, centroids, centroid_labels):  
    ## squared Euclidean distance  
    SSE = None  
    return SSE
```

Step 4 – I've helped you

```
def fit(self, X):
    ## 4) Repeat the steps 2 and 3
    ## until the cluster assignment does not change
    ## or a user-defined tolerance
    ## or a maximum number of iterations is reached

    ## step 1)
    centroids = self.initialise_centroids(X)
    self.centroids_ = np.expand_dims(np.array(centroids), axis=0)
    print(centroids)

    for n_iter in range(self.max_iter):

        ## printing progress
        sys.stderr.write('\rIteration: %d/%d' % (n_iter+1, self.max_iter))
        sys.stderr.flush()

        ## step 2)
        centroid_labels = self.assign_to_centroids(X, centroids)
        if n_iter == 0:
            self.centroid_labels_ = \
                np.expand_dims(np.array(centroid_labels), axis=0)
        ## step 3)
        centroids = self.move_centroid_to_new_centres(X, centroid_labels)
```

Step 4 – building output

```
## build kmeans object
self.cost_.append(self.calculate_cost(X, centroids,
                                     centroid_labels))

self.centroids_ = np.concatenate((self.centroids_,
                                   np.expand_dims(centroids, axis=0)),
                                   axis=0)

self.centroid_labels_ = np.concatenate((self.centroid_labels_,
                                         np.expand_dims(centroid_labels, axis=0)), axis=0)

## convergence check
## break out of loop if tol is reached
if n_iter > 1:
    diff = np.abs(self.cost_[-2] - self.cost_[-1])
    if diff < self.tol:
        break
```

Fit on the same data used in the chapter

```
from sklearn.datasets import make_blobs

X, y = make_blobs(n_samples=150,
                  n_features=2,
                  centers=3,
                  cluster_std=0.5,
                  shuffle=True,
                  random_state=0)

km = kmeans(k=3)

km.fit(X)

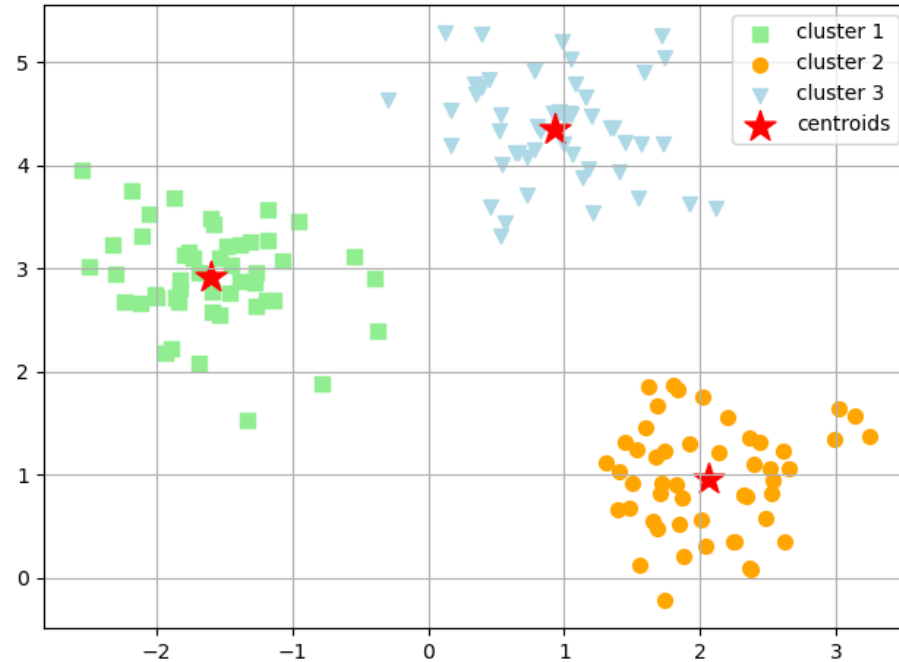
y_km = km.centroid_labels_[-1, :] ## pick last one (converged one)
cluster_centers = km.centroids_[-1, :, :] # pick last one (converged one)
```

Plot clusters

```
import matplotlib.pyplot as plt

plt.figure()
plt.scatter(X[y_km == 0, 0],
            X[y_km == 0, 1],
            s=50,
            c='lightgreen',
            marker='s',
            label='cluster 1')
plt.scatter(X[y_km == 1, 0],
            X[y_km == 1, 1],
            s=50,
            c='orange',
            marker='o',
            label='cluster 2')
plt.scatter(X[y_km == 2, 0],
            X[y_km == 2, 1],
            s=50,
            c='lightblue',
            marker='v',
            label='cluster 3')
plt.scatter(cluster_centers[:, 0],
            cluster_centers[:, 1],
            s=250,
            marker='*',
            c='red',
            label='centroids')

plt.legend()
plt.grid()
plt.tight_layout()
plt.show()
```



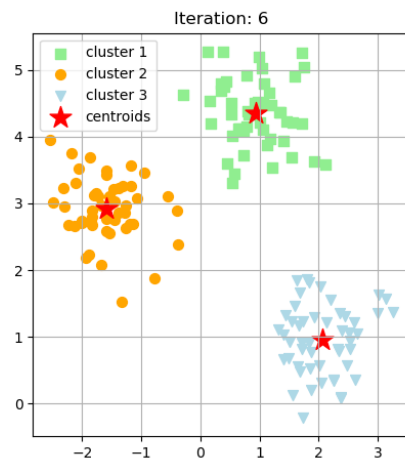
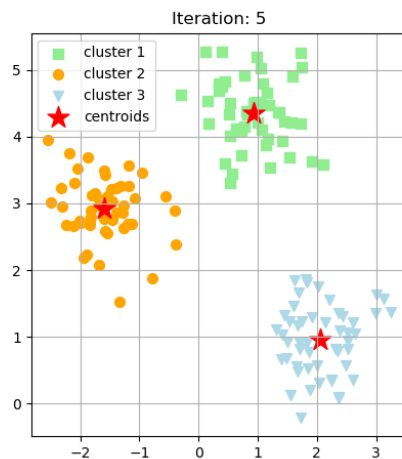
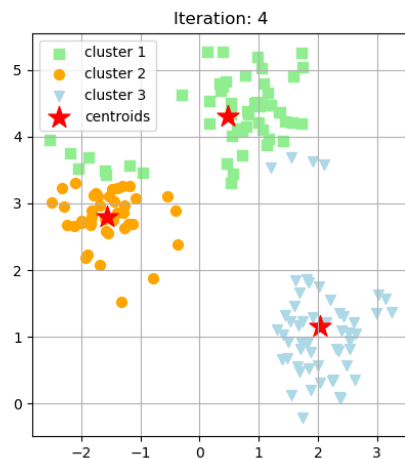
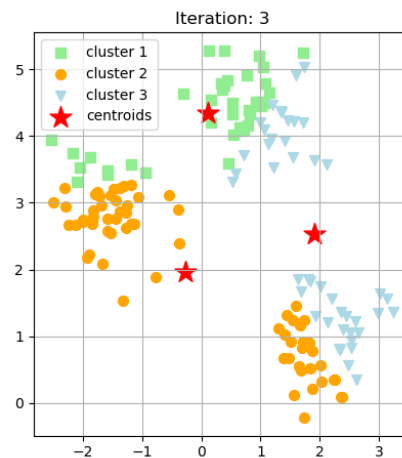
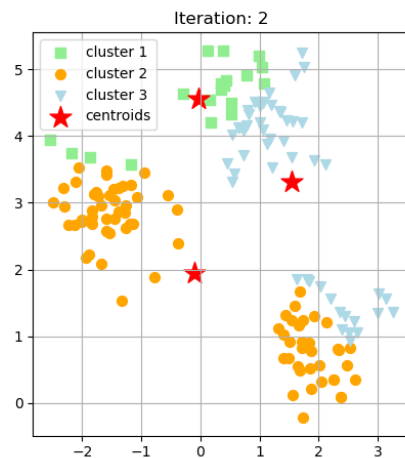
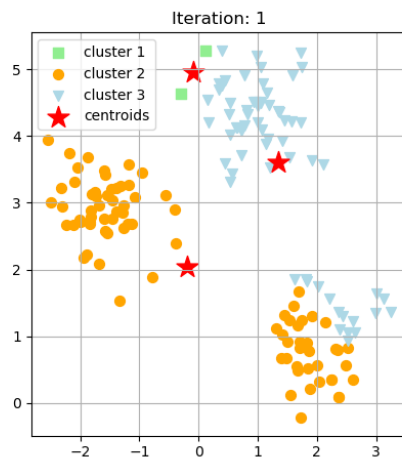
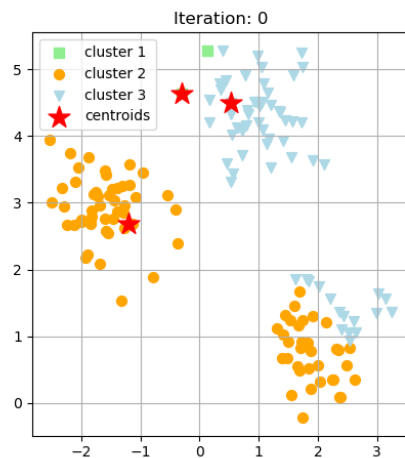
Instructions

K-means ▼



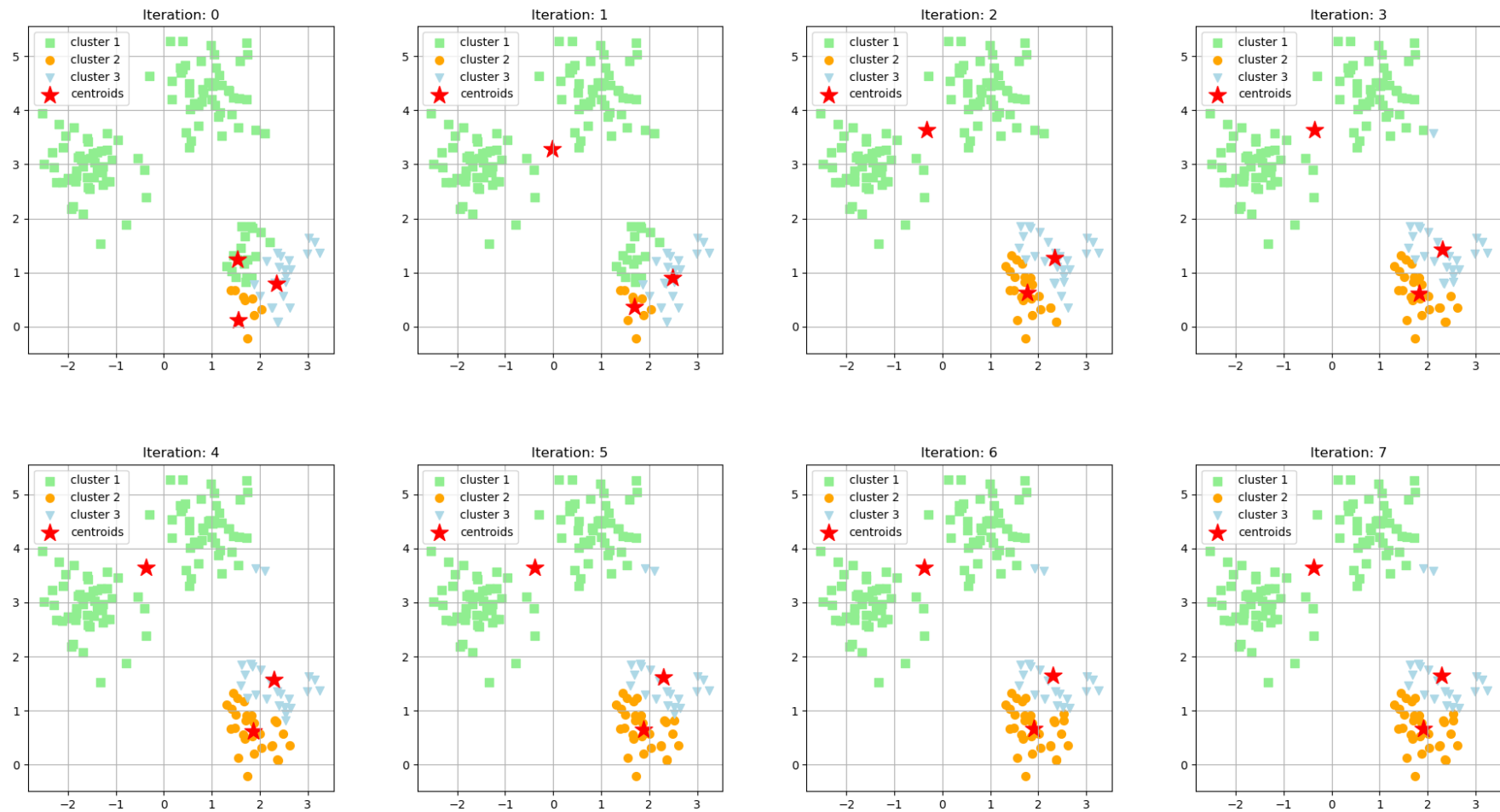
Starts 26 Nov

- Find it on BrightSpace
- Work in pairs or triplets
- If you *finish*, please seek out a group who hasn't *finished* and help them, without (!) bringing your own computer, i.e. help them on their terms and help them *finish*
- Before we *finish*, a *finished* group will present their solution to everyone
- Even if you *finish* the assignment correctly, some of your fits may *finish* badly, due to unlucky initialisation
- If you finish early, try to plot silhouette plots and elbow plots

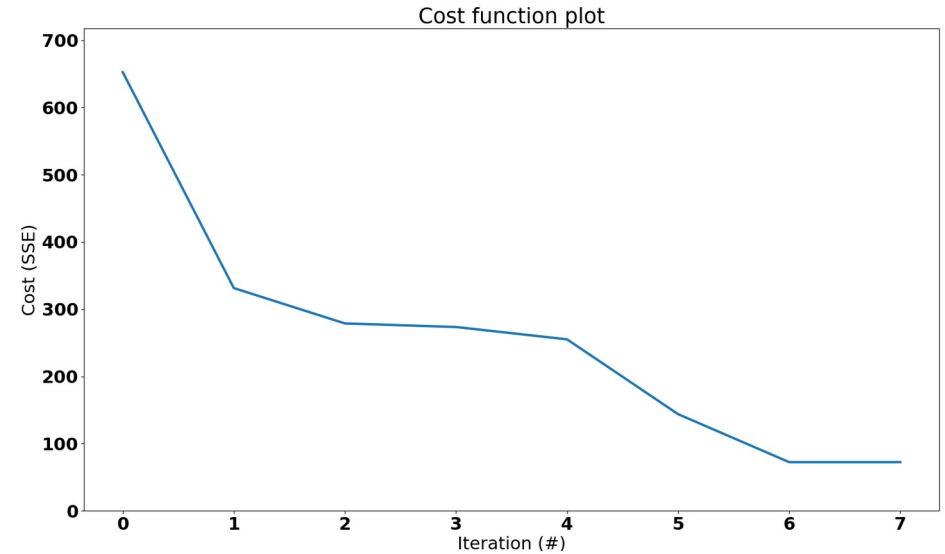
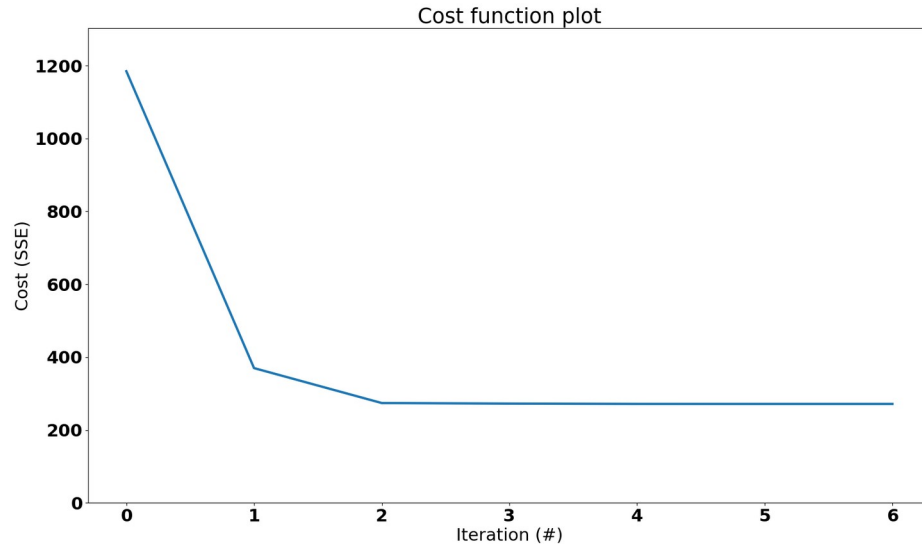


Progression of fit

But starting too close together may still cause problems (remember *k*-means++ may help)



Which one do you think is the bad fit?



Summary

- k -means is a versatile tool that can help you find patterns in many-dimensional datasets
 - we have just looked at two dimensions, but you can extrapolate
- As always – do check whether it fits reasonably
 - i.e. do you get similar costs on multiple runs

The course plan

Week 37:

Lesson 0: What is it all about?

Class 0: Setting up R and Python — and recollection of the general linear model (Lau)

Week 38:

Lesson 1: Multilevel linear regression

Class 1: Modelling subject level effects – and how do they differ from group level effects? (Lydia)

Week 39:

Lesson 2: Beyond linear regression — link functions

Class 2: What to do when the response variable is not continuous? (Lau)

Week 40:

Lesson 3: Evaluating generalised linear mixed models — and the spectre of collinearity

Class 3: Code review (Lydia)

Week 41:

Lesson 4: Explanation versus prediction

Class 4: Summarising mixed models (Lydia)

Week 42:

no teaching (go fetch potatoes instead)

Week 43:

Lesson 5: Mid-way evaluation and machine learning introduction

Class 5: Getting Python running (Lau)

Week 44:

Lesson 6: Linear and logistic regression revisited (machine learning)

Class 6: Categorizing responses based on informed guesses (Lau)

Week 45:

Lesson 7: Model evaluation and hyperparameter tuning

Class 7: Code review (Lydia)

Week 46:

Lesson 8: Dimensionality reduction — principled component analysis (PCA)

Class 8: What to do with very rich data? (Lau)

Week 47:

Lesson 9: Neural networks

Class 9: Code review (Lau)

Week 48:

Lesson 10: Unsupervised classification

Class 10: Create video explainers of central concepts (shared resource) (Lydia)

Week 49:

Lesson 0 again: What was it all about? and final evaluation

Class 11: Summarising machine learning (Lydia)

Week 50:

Student presentations:

Class 12: Work on portfolios: Ask anything! (Lydia & Lau)

Learning goals and outline

Unsupervised classification

- 1) Understanding the *k*-means algorithm
- 2) Understanding the difference between unsupervised and supervised learning
- 3) Being able to build your own *k*-means algorithm

Next time

- What have we learnt?

Reading questions

- Go through the slides from each of the lectures so far